PEC 1

Fecha límite: 26/10/26

1. Se está probando un sistema distribuido para gestionar la cita previa y el historial clínico en la red de ambulatorios de tres ciudades. El sistema incluye un servicio de gestión de citas y una base de datos replicada entre las tres ubicaciones. Se pide realizar un estudio técnico y un diseño preliminar del sistema. Responde a los siguientes apartados:
    a. Identifica dos recursos compartidos y un ejemplo concreto de concurrencia en el sistema.

    RESPUESTA
    -Recursos compartidos podrían ser dos usuarios intentando reservar una cita exactamente a la misma hora, o dos médicos editando al mismo tiempo la misma ficha médica.
    -Como ejemplo de concurrencia del sistema sería cuando un usuario desde una web y un administrativo desde un mostrador de un ambulatorio intentan reservar una cita con el mismo doctor en la misma franja libre a la vez.
    Para mitigar posibles problemas como bloqueos, doble asignación, sobreescrituras etc. podríamos añadir mecanismos de control de concurrencia como transacciones.

    REFERENCIAS
    (Voy a usar el libro DISTRIBUTED SYSTEMS Concepts and Design Fifth Edition, aunque como está en inglés lo he escaneado con un visor OCR y traducido con Machine Translation a español, he revisado las páginas y creo que coinciden con la versión en inglés)
Sección 1.4 y sección 1.5.6
Páginas 14-16, 22-23

    b. Elige cuatro tipos de transparencia de los ocho listados en la sección 1.5.7 del libro de referencia (acceso, ubicación, concurrencia, replicación, fallos, movilidad, rendimiento, escalado). Para cada una, explica cómo se manifestaría en este sistema y qué ocurriría si no se garantizara.

    RESPUESTA
    Acceso:
    -Manifestación en el sistema: Permite que se pueda acceder de igual manera desde cualquier sitio como un servidor del ambulatorio o un nodo central de otra ciudad.
    -Consecuencias si no se garantiza: El software del cliente debería de gestionar manualmente la heterogeneidad de acceso para abstraer las diferencias de sintaxis, interfaces, protocolos en función de la procedencia del dato.

    Ubicación:
    -Manifestación en el sistema: Reemplazamos la ubicación, dirección IP, o máquina desde la que nos conectamos por un identificador lógico (como el id del paciente)

-Consecuencias si no se garantiza: Si no fuera así, tendríamos que indicar la dirección de red para interactuar con el sistema. Además, si el servidor cambia su dirección, los enlaces quedarían rotos.

Replicación:
-Manifestación en el sistema: Se guardan varias copias idénticas de los datos de los pacientes en tres ciudades para mitigar los fallos y reducir la latencia de lectura. La transparencia de replicación hace que los usuarios interactúen con el sistema como si existiese una única copia y abstrae la existencia de instancias múltiples y sincronizaciones por detrás.

-Consecuencias si no se garantiza: Las aplicaciones del cliente deberían conocer la existencia de cada réplica y, cada vez que se actualizase un historial, por ejemplo, el software debería de escribir 3 veces, una por cada base de datos y gestionar los fallos y diferencias de estado.

Concurrencia:
-Manifestación en el sistema: Oculta a cada usuario si se está realizando otra consulta por otro lado sobre la misma agenda simultáneamente. Cada operación se ejecuta aislada y consistentemente sin interferencias.
-Consecuencias si no se garantiza: Se producirían condiciones de carrera; sobreescritura de diagnósticos, confirmación simultánea de la misma cita a dos pacientes distintos, desincronizaciones...

REFERENCIAS
Sección 1.5.7
Págs. 23-25

c. Indica qué transparencia sería más costosa de mantener y justifica tu respuesta en términos de complejidad técnica, impacto en la experiencia de usuario y dependencia del modelo de replicación.

RESPUESTA
-La complejidad técnica más compleja de mantener sería la de replicación.
Mantener réplicas de las bases de datos en 3 ciudades muy distantes entre sí con coordinación distribuida requiere del uso de protocolos como 2PC o el algoritmo Paxos/Raft, que se dedican a recuperar el sistema ante caídas, detección de fallos, ordenación de operaciones...

-Impacto en la experiencia de usuario: Introducimos una pequeña latencia a la interfaz del usuario ya que, al introducir replicación, en cada escritura debemos esperar a recibir confirmación de los nodos de las otras ciudades.

-Dependencia del modelo de replicación:

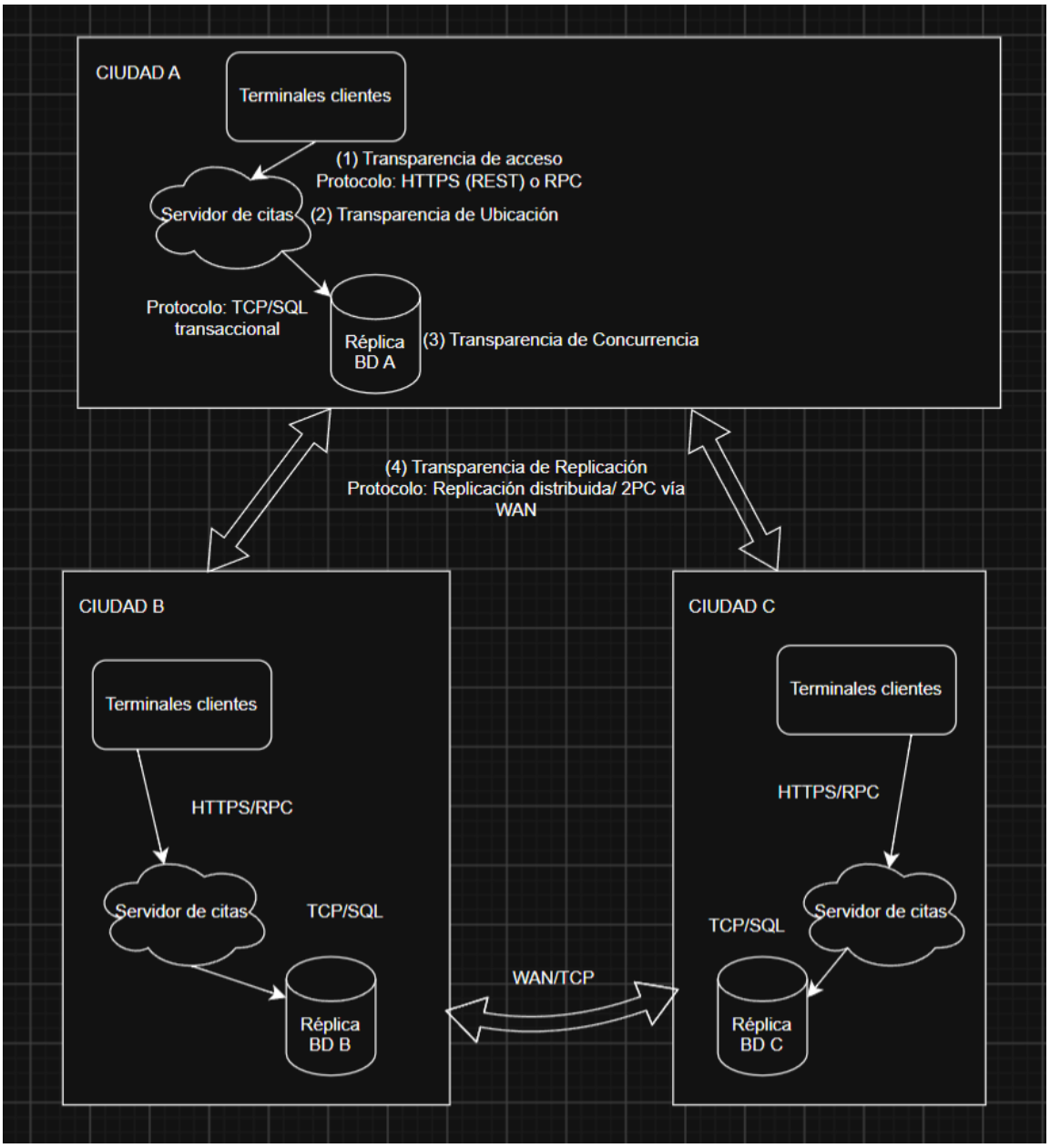
Modelo pasivo (primario-secundario) Se crea un cuello de botella donde esté el servidor maestro y se inutilizan las reservas de citas si el enlace cae.

Modelo activo (multimaestro) Las copias locales de cada ciudad son inmediatas, pero sostener la transparencia de replicación requiere un complejísimo mecanismo de resolución de conflictos

REFERENCIAS
Secciones 1.5.4, 1.5.5, 1.5.7 + Capítulo 17
Páginas. 19-25

d. Dibuja un esquema de la arquitectura del sistema que incluya:
   i. Los nodos (clientes, servidores de citas, base de datos replicada) y su ubicación en las tres ciudades.
   ii. Las comunicaciones entre ellos (protocolo o tipo de mensaje).
   iii. Sobre el propio dibujo, señala con marcas o números dónde se aplica cada una de las transparencias elegidas en el apartado b.

RESPUESTA



REFERENCIAS

Apartados 1.4, 2.3.1 y 2.3.2.

e. Explica qué ocurre en tu diseño cuando se pierde la conexión con una de las ciudades:
   i. Qué funcionalidades siguen operativas.
   ii. Qué funcionalidades dejan de estar disponibles.
   iii. Qué transparencia se ve afectada y por qué.

RESPUESTA
Vamos a asumir que se aísla la ciudad 3/C por un fallo del enlace WAN.
-Funcionalidades que siguen operativas:
En la ciudad aislada funciona la copia local de la base de datos y la atención médica presencial de una manera degradada.
En las otras ciudades siguen funcionando la mayoría de los componentes y se puede seguir tramitando citas y actualizando historiales y sincronizándose las dos entre sí.

-Funcionalidades que dejan de estar disponibles:
En la ciudad aislada no se puede concertar cita con médicos de las otras ciudades.
Tampoco se puede replicar los historiales clínicos en el resto de la red, ni se puede consultar la información actualizada en tiempo real de las Ciudades 1 y 2.

-Transparencia afectada y por qué:
Se ve comprometida la transparencia de fallos. El usuario ve excepciones, bloqueos por timeout o avisos de servicio no disponible.
También se rompe la transparencia de replicación. Los usuarios se dan cuenta de que la copia local está aislada respecto del resto

REFERENCIAS
Sección 1.5.5 y 1.5.7
Páginas 21-24

2. Observa el siguiente diagrama de una arquitectura distribuida y responde a las preguntas:
   a. Según la clasificación del libro de referencia, ¿se trata de una arquitectura de dos o de tres capas? Justifica tu respuesta identificando la función de cada capa.

RESPUESTA
Se trata de una arquitectura de 3 capas.
-Capa de presentación: Interacciona directamente con el usuario, captura peticiones y entrega contenidos
Se compone del DNS (Route 53), el balanceador de carga (LB) y el CDN.

-Capa de procesamiento/aplicación: Ejecuta la lógica de negocio, orquesta las operaciones del sistema, transforma las peticiones de los clientes y coordina el

acceso a datos.
Se compone de los servidores tanto frontales (Web service) cono los de aplicación (App Server) en el bloque AppTier.

-Capa de datos: Es la capa de persistencia y almacenamiento a largo plazo. Supongo que la caché servirá como estrategia Cache-aside en la que la cache posibilita una lectura rápida y evita tantas lecturas lentas a una base de datos SQL.
Se compone de una caché como memoria intermedia rápida, la base de datos relacional (RDB), distribuida en varias zonas de disponibilidad y la base de datos NoSQL, la cual está conectada directamente con la CDN.

REFERENCIAS
Sección 2.3.2
Páginas 52-54

b. ¿Qué aporta que los servicios web puedan escalar automáticamente? Explica ante qué cambios en la carga sería útil esta capacidad.

RESPUESTA

-Permite adaptar el sistema al volumen de tráfico añadiendo o retirando instancias de los servicios web en tiempo de ejecución en función de las métricas y monitorizaciones. Así, evitamos cuellos de botella, latencias innecesarias...
-Permite optimizar los costes para la demanda real eliminando los servidores ociosos y adaptando las infraestructuras sobredimensionadas
-Tolerancia a fallos: Si un servidor se cuelga, el grupo lo reemplaza automáticamente.

-Es útil ante picos imprevistos de demanda, como un contenido viral en redes, que podría provocar denegación de servicio.
-Ante patrones de carga estacionales/periódicos: Por ejemplo, ante un partido de fútbol, o que haya mayor tráfico en horario nocturno.
-Crecimiento continuo de la base de usuarios: Sin requerir rediseños del software.

REFERENCIAS
Sección 1.3.4, 1.5.4, 1.5.8
Páginas 13-14, 19-21, 25-26.

c. ¿Qué función cumple el enrutamiento DNS en esta arquitectura? En AWS, este servicio se ofrece mediante Amazon Route 53.

RESPUESTA
-Proporciona nombres de dominio amigables para los clientes. Así no necesitan conocer la dirección IP ilegible de los servidores.
-El DNS redirige el tráfico hacia el balanceador de carga y hacia la CDN.
-También, si detecta una interrupción en una zona de disponibilidad, actualiza sus registros para encaminar a los usuarios a recursos activos.

-Puede enrutar hacia la región o centro de datos más próximos en términos de latencia de red.

REFERENCIAS
Sección 1.5.7. Capítulo 13.2
Páginas 23-25, 569-584

d. ¿Qué aportan las distintas zonas de disponibilidad en las que se despliegan los recursos? Relaciona su uso con los retos de diseño de un Sistema Distribuido (sección 1.5 del libro de referencia)

RESPUESTA

-Tratamiento y tolerancia a fallos: Si un nodo cae en la Zona de disponibilidad A, la zona de disponibilidad B asume el rol primario, enmascarando los fallos

-Escalabilidad: Ante posibles cuellos de botella de rendimiento, permite distribuir la carga en diferentes ubicaciones físicas para que el sistema no colapse.

-Concurrencia: Replicamos las bases de datos entra la zona A y B. Nuestro reto es garantizar que no se produzcan escrituras contradictorias ni estados inconsistentes entre réplicas al realizar transacciones.

-Calidad de servicio: La caché de la Zona A, coordinada con la replicación hacia la zona B y la salida hacia la CDN garantizan cumplir con una transmisión de contenido multimedia en tiempos óptimos.

REFERENCIAS

Sección 1.5.5, 1.5.4, 1.5.6, 1.5.8

Páginas 16-26

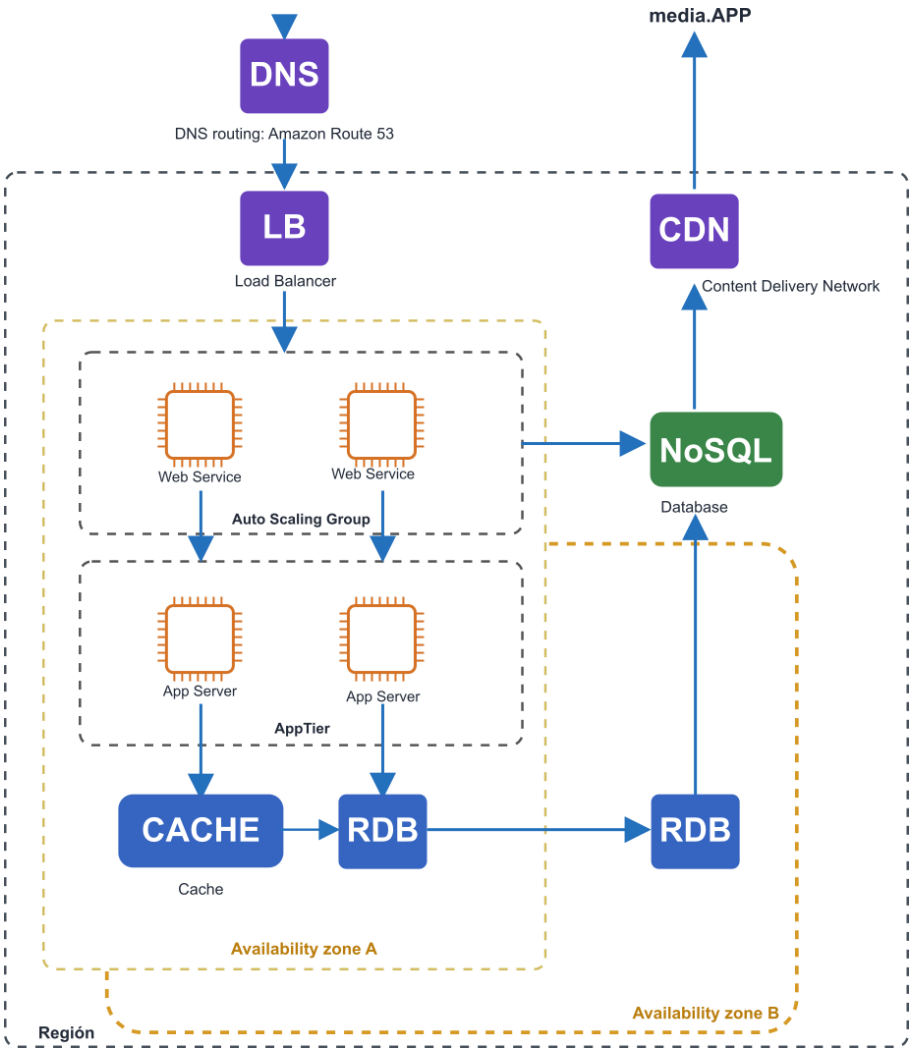


Ilustración 1. Arquitectura distribuida. Nótese que la aplicación tiene como finalidad servir contenido multimedia, que se entrega al usuario a través de una CDN utilizando el almacenamiento como origen.

3. Un servicio de reservas replicado en dos centros de datos usa un detector de fallos basado en *heartbeats* y *timeouts*. Analiza los siguientes aspectos:

    a. Clasifica según el modelo de fallos del libro (parada, omisión, arbitrarios/bizantinos, temporización) los siguientes casos: un servidor que se cuelga, un mensaje que se pierde, una réplica que responde con datos corruptos y una respuesta que llega con mucho retraso.

RESPUESTA

-Un servidor que se cuelga: Fallo de omisión en proceso.
El servidor suspende su actividad de forma abrupta e indefinida, y omite realizar las acciones que tenía programadas. Si permitiese al resto de nodos constatar que se ha detenido, se llamaría fail-stop.

-Un mensaje que se pierde: Fallo de omisión en canal de comunicación.
El emisor deposita el mensaje en su búfer de salida, pero el canal no lo transporta hasta el búfer de entrada del receptor (normalmente por congestión de red). El canal omite la entrega de información.

-Una réplica que responde con datos corruptos: Fallo arbitrario o bizantino.
Peor tipo de fallo posible. El proceso réplica no se detiene ni omite la comunicación, sino que realiza una acción errónea/imprevista. Ocurre porque procesa o devuelve un resultado incorrecto en su carga útil.

-Una respuesta que llega con mucho retraso: Fallo de temporización en un sistema síncrono (o no clasificable en uno asíncrono).
En un sistema síncrono, si la respuesta supera el límite de tiempo establecido, se califica como un fallo de rendimiento.
En un sistema asíncrono, no existen garantías ni límites de tiempo prefijados.

REFERENCIAS

Secciones 2.4.1 y 2.4.2

Páginas 64-71.

    b. Explica por qué, en un sistema asíncrono, no se puede distinguir con certeza un servidor caído de uno simplemente lento. Analiza las consecuencias de esta incertidumbre al elegir el valor del *timeout*.

RESPUESTA

-Un sistema asíncrono se caracteriza porque no existen niveles superiores sobre la velocidad de ejecución de los procesos, los retrasos de transmisión de los mensajes y la tasa de deriva de los relojes locales de cada nodo.
-No se puede distinguir con certeza un servidor caído de uno lento en un sistema asíncrono porque, con un detector de fallos basado en heartbeats y timeouts, si dejamos de recibir señales de un servidor y el contador vence, se produce una indeterminación fundamental: o el servidor ha caído o está operativo pero saturado.
-Como los retrasos pueden ser enormes sin violar las premisas del modelo, ningún timeout finito permite estar seguros acerca de si el servidor ha caído o va lento.

-Si se elige un timeout demasiado corto
Parte positiva: Detección rápida de fallos reales.
Parte negativa: Se pueden dar falsos positivos cuando se puede tratar de una congestión de la red

-Si se elige un timeout demasiado largo
Parte positiva: Alta precisión y eliminación de falsas alarmas al dar un largo margen temporal.
Parte negativa: Si sufrimos una caída real, el sistema permanecerá bloqueado esperando una respuesta.

REFERENCIAS

Secciones 2.4.1 y 2.4.2

Páginas 64-71.

4. La búsqueda de inteligencia extraterrestre (*Search for Extraterrestrial Intelligence, SETI)* comprende distintas iniciativas científicas que analizan señales astronómicas en busca de posibles indicios de actividad tecnológica. Una de ellas fue SETI@home (iniciada en el 1999), que permitió a personas de todo el mundo aportar la capacidad de cálculo de sus ordenadores. Para coordinar ese trabajo se utilizó BOINC (Berkeley Open Infrastructure for Network Computing). https://boinc.berkeley.edu/

   Investigad cómo funcionaba SETI@home mediante BOINC y responded a las siguientes preguntas:

   a. ¿Por qué podemos considerarlo un sistema distribuido? Identificad sus componentes y explicad cómo se repartían las tareas y se recogían los resultados.

RESPUESTA
-Un sistema distribuido es aquel en el que los componentes hardware o software ubicados en ordenadores conectados en red se comunican y coordinan sus acciones únicamente mediante el paso de mensajes.
-En el caso de SETI@home en BOINC tiene
1) Concurrencia de componentes: Miles de ordenadores dispersos geográficamente ejecutan procesos de forma simultánea e independiente.
2) Ausencia de reloj global: La sincronización se resuelve mediante mensajes asíncronos de petición y entrega de trabajo.
3) Fallos independientes: La desconexión de un nodo voluntario no detiene la ejecución del proyecto ni frena el progreso del resto de participantes

Por lo tanto, es un sistema distribuido.

-Componentes del sistema: Arquitectura master-worker.
Servidores centrales:
1.Generador de trabajo (Work Generator/Splitter): Recibimos desde telescopios

un flujo continuo de señales astronómicas. Este software lo divide en bloques de datos independientes llamados unidades de trabajo
2.Planificador (Scheduler): Asigna las unidades de trabajo, controla las deadlines y gestiona las peticiones de los clientes.
3.Validador: Compara los resultados de distintos voluntarios para la misma tarea y verifica que sean concordantes y no tengan datos corruptos.
4.Asimilador: Introduce los datos validados en la BBDD de investigación.
Nodos clientes: Ordenadores personales con el software BOINC como proceso en segundo plano
-Reparto de tareas y recolección de resultados: Basado en un modelo pull.
1. Solicitud y reparto: El cliente BOINC voluntario solicita trabajo al servidor. El servidor envía workunits junto con otros parámetros como el deadline de entrega. Si es la primera vez que el voluntario ejecuta en esta plataforma, descarga los binarios del algoritmo científico
2. Procesamiento: El cliente analiza la señal en los ciclos ociosos de su CPU/GPU
3. Recolección y validación: Termina el procesamiento. El cliente realiza un POST HTTP al servidor y sube el resultado en un archivo. El servidor marca la tarea como recibida. El validador verifica la workunit comparándola con la de otros voluntarios. Si son idénticas, almacena las conclusiones. Si expira la deadline, el servidor descarta el intento y reasigna la tarea a otro voluntario.

REFERENCIAS
Artículo de Wikipedia sobre el funcionamiento de BOINC
https://de.wikipedia.org/wiki/Berkeley_Open_Infrastructure_for_Network_Computing

boinc.berkeley.edu

https://github.com/BOINC/boinc/wiki/How-BOINC-works

Sección 1.1, 1.2, 1.4, 2.3.1.

Págs. 2-5, 14-16, 45-48

b. Comparad esta forma de computación voluntaria con una solución equivalente desplegada en la nube. ¿Qué diferencias encontraríais en cuanto al control sobre los recursos, la disponibilidad, la tolerancia a fallos, el coste y la escalabilidad?

RESPUESTA

Criterio de comparación	Voluntaria	Cloud Computing
Control sobre los recursos	Entre nulo o bajo	Control total
Disponibilidad	Baja e impredecible	Alta y garantizada por contrato
Tolerancia a fallos	Diseñada para fallos de omisión y bizantinos	Diseñada para fallos de parada
Coste	Prácticamente nulo	Pago por uso
Escalabilidad	Dependiente de la repercusión social, pero masiva hipotéticamente	Elástica, rápida, multipropósito

REFERENCIAS

Sección 1.3.4, 1.5.3, 1.5.4, 1.5.5 y 2.4.2

Páginas 13-22, 69-71

5. Una aplicación de telemedicina envía resultados de análisis entre el móvil del paciente y el servidor del hospital.

    a. Elabora un modelo de amenazas: identifica a los posibles adversarios y cita cuatro amenazas (escucha, suplantación, manipulación, repetición, denegación de servicio), indicando el recurso afectado.

RESPUESTA

Posibles adversarios:
-Intruso pasivo(eavesdropper): Alguien que se sitúa en nuestra misma red, la cual no tiene cifrado adecuado, y escucha el tráfico para acceder a datos médicos confidenciales
-Atacante activo (man-in-the-middle): Toma el control de los nodos de red y puede modificar resultados analíticos o suplantar identidades.
-Operador de botnets: Un atacante externo que busca saturar la infraestructura hospitalaria.

1)Escucha (Eavesdropping): Vulnera la confidencialidad al capturar copias de los paquetes de datos.
Recurso afectado: Los datos médicos y resultados de análisis de los pacientes

2)Suplantación de identidad (Masquerading/Spoofing): Vulnera la autenticidad y el control de acceso. El atacante se hace pasar por el móvil del paciente legítimo (falsificando identificadores) o suplanta al servidor del hospital.
Recurso afectado: Servidor del hospital o la cuenta del paciente.

3)Manipulación/alteración de mensajes (Tampering): Vulnera la integridad del mensaje. Consiste en interceptar los mensajes con datos médicos mientras viajan por el canal, alterarlos y reenviarlos.
Recurso afectado: Resultados de análisis clínicos.

4)Denegación de servicio (DDOS): Vulnera la disponibilidad al bombardear el servidor del hospital con una avalancha de peticiones.
Recurso afectado: Ancho de banda del servidor del hospital.

REFERENCIAS

Sección 2.4.3, 1.5.3, y 11.1

Páginas 18-19, 71-76 y 464-472

b. Explica cómo un canal seguro (TLS) mitiga cada amenaza. Distingue el papel de la criptografía simétrica, la asimétrica, las firmas digitales y los certificados.

RESPUESTA
-Un canal seguro con protocolo TLS consiste en una abstracción construida sobre la capa de transporte TCP que garantiza autenticación, privacidad e integración entre procesos. Mitiga cada amenaza de esta forma:
1)Contra la escucha: Cifrando todos los datos que viajan por el canal.
2)Contra la suplantación: Autentica todos los extremos.
3)Contra la manipulación: Añade funciones resumen a los bloques de datos, descartando los mensajes que no tengan códigos de autenticación criptográficos como HMAC, AEAD, AES-GCM.
4)Contra la denegación de servicio: Mitiga las peticiones falsas no autenticadas, aunque debe ser complementado con cortafuegos, por ejemplo.

-Criptografía simétrica: Usa una única clave compartida para cifrar y descifrar. Después del handshake, se cifran todos los datos con algoritmos como AES, más rápidos y eficientes que los asimétricos

-Criptografía asimétrica: Consiste en una clave pública y otra privada. Se usa en la fase inicial. Permite al cliente comprobar la identidad del hospital y acordar una clave secreta compartida de sesión no transmitida a través de la red.

-Firmas digitales: Se general hasheando un resumen del mensaje con el cifrado de dicho resumen con la clave privada del emisor. Legitiman el certificado del emisor.

-Certificados digitales: Los emite una Autoridad de Certificación de confianza para vincular la identidad del hospital con su clave pública. Evitan Man-in-the-Middle.

REFERENCIAS

Sección 2.4.3, 11.2, 11.3, 11.6.3

Páginas 74-75, 472-478, 484-496, 510-515

c. Describe un ataque de repetición (*replay*) y cómo se evita mediante *nonces* o marcas de tiempo.

RESPUESTA
Consiste en interceptar una copia de un mensaje válido, la cual no puede leer ni descifrar el atacante, y spamearlo de nuevo sin modificarlo para contaminar, por ejemplo, en este caso, el historial médico.
Se previene con marcas de tiempo en la carga del mensaje al emitir la analítica,

para que el servidor del hospital la compare con su propio reloj, y si supera un cierto umbral, descartar el mensaje por obsoleto.

También se puede usar Nonces, que son valores aleatorios que son generados para una única interacción y que jamás se repite. Tiene la ventaja de que no necesita de sincronización de relojes entre equipos. Supongo que será útil para sistemas asíncronos.
Funciona así: el cliente (móvil del paciente) solicita un reto al servidor. El móvil empaqueta la payload con el nonce y lo devuelve. El servidor verifica la respuesta y tras validarlo, borra e invalida N para futuras respuestas.

REFERENCIAS

Sección 2.4.3, 11.2

Páginas 74-75, 474-476

d. Explica qué problema resuelve un servicio de autenticación como Kerberos frente a que cada servicio gestione sus propias claves de manera independiente.

RESPUESTA
-La gestión independiente de claves tiene varios problemas: Cada servicio independiente puede tener su propio servidor/repositorio
1) Inviabilidad computacional [O(N*M)]: Donde N son los pacientes y M los servicios hospitalarios, que necesitan administrar NxM claves secretas compartidas
2) Exposición de secretos en la red: Hay que transmitir claves secretas e identificadores por la red a cada sevicio independiente, lo que multiplica las oportunidades de ataque
3) Problemas de coherencia y revocación: Cada cambio obliga a sincronizarlo en todos los servidores. Si uno de ellos no actualiza correctamente, se genera una brecha de seguridad

-Kerberos (Trusted Third Party) aporta:
1) Single Sign-On(SSO): El usuario se identifica una vez en el Servidor de Autenticación (AS).
2) Uso de Tickets cifrados y Keys efímeras: La clave secreta permanente nunca se envía por la red, sino una clave de sesión temporal generada por Kerberos. Kerberos emite Tickets temporales cifrados a través del Ticket Granting Service (TGS). Si el usuario quiere consultar su historial, presenta un ticket para el servicio en particular.
3) Gestión y revocación centralizada: La complejidad se reduce a [O(N+M)] porque solo se comparte una clave con el servidor Kerberos. Los cambios se aplican instantáneamente y de manera centralizada a todos los servicios de la red.

REFERENCIAS
Sección 11.2, 11.6
Páginas 473-478, 503-510